



FINAL YEAR ENGINEERING PROJECT

Smart Medicine Availability & Intelligent Janaushadhi Recommendation System



FRONTEND ARCHITECTURE

Project	Smart Medicine Availability & Intelligent Janaushadhi Recommendation System
Document	FRONTEND ARCHITECTURE
Version	v1.0.0
Date	02 July 2026

Smart Medicine System — React Architecture Guide

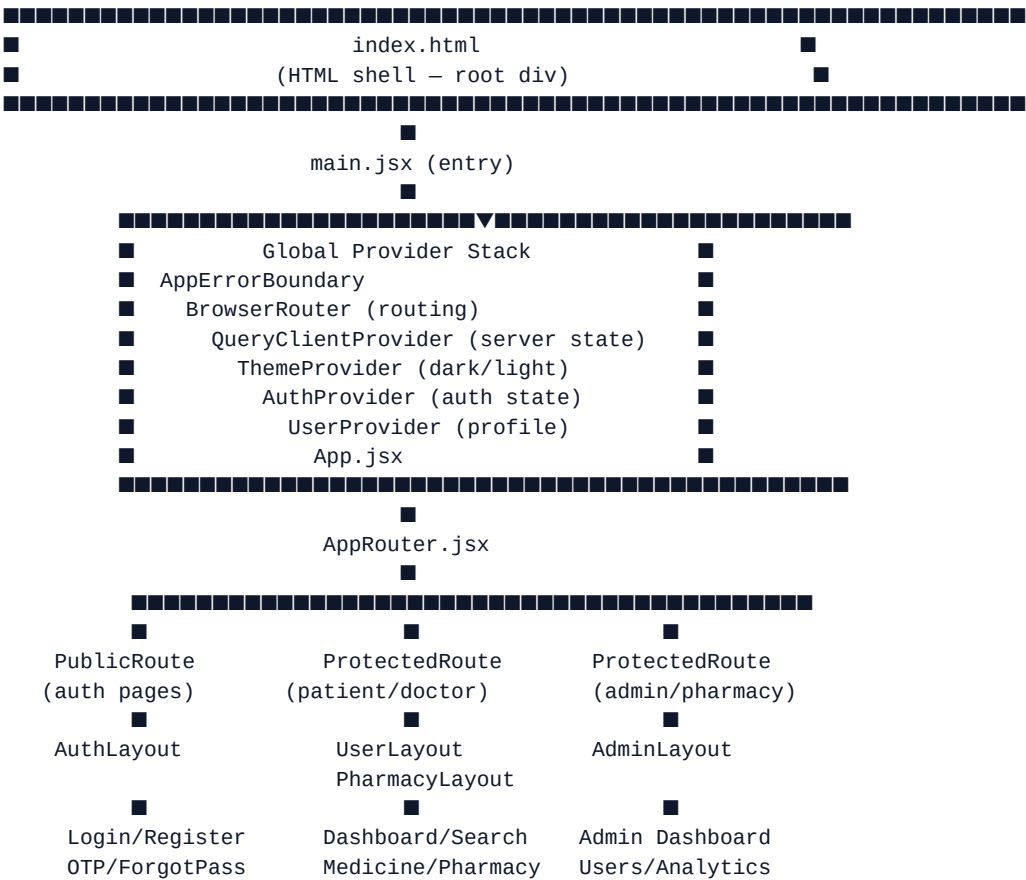
1. Why This Architecture?

A React project can technically be built by putting everything in one file. But as an app grows, that becomes impossible to maintain. This project uses **industry-standard React architecture** — the same patterns used at companies like Airbnb, Shopify, and startups worldwide.

The architecture follows three guiding principles:

- 1. **Separation of Concerns** — each file does one job
- 2. **Reusability** — write once, use everywhere
- 3. **Scalability** — adding new features should not require touching old code

2. Architecture Overview Diagram



3. Layer by Layer Explanation

Layer 1: Entry Point (`main.jsx`)

This is where the React application starts. It:

- Creates the React root (mounts React into `<div id="root">` in index.html)
- Wraps the app in all global **providers** in the correct order
- Sets up the toast notification container

Why this order?

- `BrowserRouter` must wrap everything because routing hooks are used inside contexts
- `QueryClientProvider` must wrap `AuthProvider` so queries can be cleared on logout
- `AuthProvider` must wrap `App` because `App` reads auth state

Layer 2: App Component (`App.jsx`)

The first React component that consumes providers. It:

- Shows `<OfflinePage>` if the browser is offline
- Shows a loading spinner during auth rehydration on page refresh
- Renders `<AppRouter>` when ready

Layer 3: Router (`routes/`)

Three files handle all routing:

File	Purpose
AppRouter.jsx	Defines all routes with lazy loading
ProtectedRoute.jsx	Blocks unauthenticated users → redirect to /login
PublicRoute.jsx	Blocks authenticated users from auth pages → redirect to dashboard

Layer 4: Layouts (`layouts/`)

Layouts are **persistent shells** that wrap pages. The page content changes, but the navigation stays.

Layout	Used For
MainLayout	Public home page — has Navbar and Footer
AuthLayout	Login/Register pages — centered card, no nav
UserLayout	Patient dashboard pages — Sidebar + TopBar
PharmacyLayout	Pharmacy staff pages — Sidebar + TopBar
AdminLayout	Admin portal pages — Sidebar + TopBar

Layer 5: Pages (`pages/`)

Pages are the actual screens users see. Each page:

- Lives in its own folder (`pages/search/MedicineSearchPage.jsx`)
- Is **lazy-loaded** (not loaded until the user navigates to it)
- Contains sections as sub-components in a `sections/` subfolder
- Has no reusable logic — all reusable logic is in components/hooks/services

Layer 6: Components (`components/`)

The heart of the UI. All components are:

- **Reusable** — used across multiple pages
- **Prop-driven** — behaviour controlled via props
- **Independent** — no direct API calls, no direct routing

Subcategories:

```

components/
  ■■■ ui/           ← Atoms: Button, Badge, Avatar, Divider, IconButton
  ■■■ forms/        ← Form controls: Input, Select, Checkbox, OtpInput
  ■■■ cards/        ← Card components: MedicineCard, PharmacyCard
  ■■■ common/       ← Shared utilities: Breadcrumb, SearchBar, Pagination
  ■■■ feedback/     ← Status UI: Spinner, Skeleton, EmptyState, ErrorState
  ■■■ dialogs/      ← Modal, ConfirmDialog
  ■■■ navigation/   ← Navbar, Sidebar, Footer, TopBar
  ■■■ layout/       ← Container, PageHeader, SectionHeader

```

Layer 7: Contexts (`contexts/`)

React Context provides **global state** that any component can access without prop-drilling.

Context	What It Stores
AuthContext	currentUser, isAuthenticated, login, logout
ThemeContext	theme (light/dark), toggleTheme
UserContext	userProfile, updateProfile

Layer 8: Hooks (`hooks/`)

Custom hooks extract **reusable stateful logic** from components.

Hook	What It Does
useDebounce	Delays a value update (used for search inputs)
useLocalStorage	Syncs state with localStorage
useOnlineStatus	Tracks browser online/offline state
useSessionGuard	Monitors inactivity for session timeout

Layer 9: Services (`services/`)

Services are the **API layer**. They contain every API endpoint call.

Service	API Endpoints
authService.js	login, register, OTP, forgot/reset password
userService.js	get/update user profile, admin user list
medicineService.js	search, get details, get alternatives
pharmacyService.js	nearby, search, get inventory
inventoryService.js	get, add, update, delete stock

Layer 10: Config (`config/`)

File	Purpose
axiosClient.js	Pre-configured Axios instance with JWT interceptors
env.js	Environment variable wrapper
queryClient.js	React Query client with default options

Layer 11: Utils (`utils/`)

Pure functions — no React, no side effects.

File	Contains
formatters.js	Currency, date, truncate, title case
validators.js	Phone, email, password, OTP validators
authSchemas.js	Zod schemas for all auth forms
storage.js	localStorage wrapper (get, set, remove, clear)

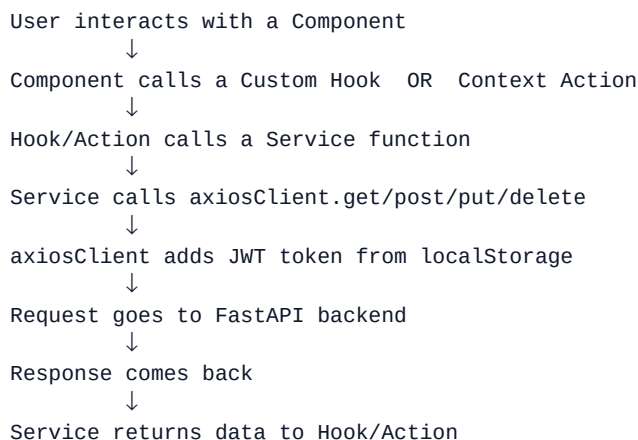
Layer 12: Constants (`constants/`)

File	Contains
app.js	USER_ROLES, STORAGE_KEYS, TOAST timings, HTTP_STATUS
routes.js	All route path strings (prevents typos)
navConfig.js	Sidebar nav items for each role

Layer 13: Styles (`styles/`)

File	Contains
index.css	Tailwind v4 @theme {} design tokens
tokens.js	JS mirror of design tokens (for charts/inline styles)
theme.js	Light/dark theme config

4. Data Flow Diagram



↓
Hook/Action updates state
↓
React re-renders the Component with new data

5. Why This Architecture Is Scalable

Problem	How This Architecture Solves It
"I need to add a new page"	Create pages/newFeature/NewPage.jsx, add route to AppRouter.jsx
"I need to change the API base URL"	Change one line in .env
"I need to add a new API call"	Add one method to the relevant service file
"I need to add a new nav item"	Add one line to navConfig.js
"I need to reuse a component"	Import from components/ — already done
"I need to change authentication"	Update AuthContext.jsx and authService.js only
"Backend is ready"	Uncomment TODO lines in services, no component changes